

PROGRAMACION I

TRABAJO PRÁCTICO INTEGRADOR

Gestion de
Datos de
Países en
Python

ALUMNO: MARTIN STERLI

AÑO: 2026

Indice

1. Marco Teorico.....	1 - 3
2. Decisiones Tecnicas y Arquitectura.....	4
3. Dificultades y Conclusiones.....	5
4. Bibliografia / Webgrafia.....	5

Marco Teorico

Listas

En Python, una lista es una estructura de datos que permite almacenar multiples elementos en una sola variable. Los elementos se guardan en orden y pueden ser de distintos tipos, como numeros, cadenas de texto o diccionarios. Se definen usando corchetes y permiten agregar, eliminar o modificar elementos en cualquier momento, lo que las hace muy utiles para manejar colecciones de datos que pueden cambiar durante la ejecucion del programa. En este proyecto se utilizo una lista llamada paises para almacenar todos los registros leidos desde el archivo CSV, donde cada elemento de la lista es un diccionario con los datos de un pais.

Diccionarios

En Python, un diccionario es una estructura de datos que almacena pares de clave y valor. A diferencia de las listas, no se accede a los elementos por su posicion sino por su clave, lo que permite buscar y modificar datos de forma mas rapida y clara. Se definen usando llaves y cada elemento tiene una clave unica asociada a un valor. En este proyecto cada pais se representa como un diccionario con cuatro claves: nombre, poblacion, superficie y continente, lo que permite acceder a cada dato de forma directa y ordenada.

Funciones

En Python, una funcion es un bloque de codigo reutilizable que realiza una tarea especifica. Se definen usando la palabra clave def seguida del nombre de la funcion y sus parametros. El uso de funciones permite dividir el programa en partes mas pequenas y organizadas, haciendo el codigo mas facil de leer, mantener y corregir. En este proyecto se aplico el principio de una funcion, una responsabilidad, donde cada funcion se encarga de una sola tarea, como agregar un pais, buscar, filtrar u ordenar. Tambien se definieron funciones auxiliares de validacion como pedir_entero_positivo y pedir_string para reutilizar la logica de control de errores en distintas partes del programa.

Condicionales

Las estructuras condicionales en Python permiten ejecutar distintos bloques de código según si una condición es verdadera o falsa. Se utilizan las palabras clave `if`, `elif` y `else` para evaluar condiciones y tomar decisiones dentro del programa. En este proyecto se utilizaron condicionales en prácticamente todas las funciones, por ejemplo para verificar si un país ya existe antes de agregarlo, para controlar que el mínimo no sea mayor que el máximo en los filtros por rango, o para mostrar un mensaje de error cuando una búsqueda no arroja resultados.

Ordenamientos

El ordenamiento es el proceso de organizar una colección de datos según un criterio específico. En Python se puede ordenar una lista usando la función `sorted()`, que recibe la lista y un parámetro `key` que indica por qué campo ordenar. También acepta el parámetro `reverse` para elegir si el orden es ascendente o descendente. En este proyecto se implementó el ordenamiento de países por nombre, población y superficie, permitiendo al usuario elegir el criterio y la dirección del orden. Se utilizó una función `lambda` como parámetro `key` para indicarle a `sorted()` qué campo del diccionario usar como criterio de comparación.

Estadísticas

Las estadísticas son cálculos que permiten resumir y analizar un conjunto de datos. En este proyecto se implementaron las siguientes estadísticas sobre el dataset de países: el país con mayor y menor población, que se obtienen recorriendo la lista y comparando los valores de población; el promedio de población y de superficie, que se calculan sumando todos los valores y dividiendo por la cantidad de países; y la cantidad de países por continente, que se obtiene usando un diccionario contador que incrementa el valor de cada continente a medida que se recorre la lista. Estos indicadores permiten tener una visión general del dataset de forma rápida y clara.

Archivos CSV

Un archivo CSV (Comma Separated Values) es un formato de texto plano que almacena datos en forma de tabla, donde cada fila representa un registro y cada columna un campo, separados por comas. Es un formato muy utilizado para intercambiar datos entre sistemas por su simplicidad y compatibilidad. En Python se puede trabajar con archivos CSV usando el módulo `csv`, que forma parte de la biblioteca estándar del lenguaje. En este proyecto se utilizó `csv.DictReader` para leer el archivo `países.csv`, que permite acceder a cada fila como un diccionario usando los nombres de las columnas como claves. Al momento de la carga se convierten los campos de población y superficie a enteros para poder realizar cálculos y comparaciones numéricas correctamente.

Decisiones Tecnicas y Arquitectura

Para el desarrollo de este proyecto se tomaron las siguientes decisiones tecnicas:

Se utilizo una lista de diccionarios como estructura principal para almacenar los paises, ya que permite acceder a cada dato por nombre de clave de forma clara y ordenada, y facilita recorrer, filtrar y ordenar los registros.

Se modularizo el codigo en funciones especificas para cada funcionalidad del menu, siguiendo el principio de una funcion, una responsabilidad. Esto hace que el codigo sea mas facil de leer, mantener y corregir.

Se definieron funciones auxiliares de validacion reutilizables, como pedir_entero_positivo y pedir_string, para evitar repetir la logica de control de errores en cada parte del programa.

Para el ordenamiento se utilizo la funcion sorted() con el parametro key y funciones lambda, lo que permite ordenar por cualquier campo del diccionario sin necesidad de implementar un algoritmo de ordenamiento manual.

El archivo CSV se lee al inicio del programa usando el modulo csv de la biblioteca estandar de Python, convirtiendo los campos numericos a enteros para poder operar con ellos correctamente.

El flujo principal del programa es un bucle while que muestra el menu y llama a la funcion correspondiente segun la opcion elegida por el usuario, manteniendose activo hasta que el usuario elija salir.

Menu principal del programa

```
Se cargaron 10 paises correctamente.

1. Agregar pais
2. Actualizar pais
3. Buscar pais
4. Filtrar paises
5. Ordenar paises
6. Estadisticas
7. Mostrar todos
8. Salir

Elige una opcion:
```

Estadisticas generales del dataset

```
Elige una opcion: 6

////////// ESTADISTICAS //////////

Pais con mayor poblacion: China (141260000)
Pais con menor poblacion: Australia (25921000)
Promedio de poblacion: 235058374.60
Promedio de superficie: 4274465.60 km2

Cantidad de paises por continente:
America: 4
Asia: 2
Europa: 2
Oceania: 1
Africa: 1

Presiona Enter para volver al menu..
```

Resultado de busqueda por nombre parcial

```
1. Agregar pais
2. Actualizar pais
3. Buscar pais
4. Filtrar paises
5. Ordenar paises
6. Estadisticas
7. Mostrar todos
8. Salir

Elige una opcion: 3

Ingresa el nombre o parte del nombre a buscar: ar

//////// Resultados para 'ar' //////////

Nombre: Argentina | Poblacion: 45376763 | Superficie: 2780400 km2 | Continente: America

Presiona Enter para volver al menu..
```

Dificultades y Conclusiones

Durante el desarrollo del proyecto surgieron algunas dificultades. Una de las principales fue el manejo de errores en las entradas del usuario, ya que habia que contemplar tanto valores no numericos como valores fuera de rango en distintas partes del programa. Esto se resolvió creando funciones auxiliares de validacion reutilizables que centralizan ese control y evitan repetir código.

Otra dificultad fue el manejo del archivo CSV, especialmente la conversion de los campos numericos al momento de la lectura, ya que por defecto todos los valores se leen como texto y era necesario convertirlos a enteros para poder realizar calculos y comparaciones.

En cuanto a las conclusiones, este proyecto permitio afianzar el uso de listas y diccionarios como estructuras principales para organizar datos, y entender la importancia de modularizar el código en funciones con una responsabilidad clara. También se pudo aplicar de forma practica el manejo de archivos CSV y el control de errores con try/except, conceptos que resultan fundamentales en cualquier programa real que trabaje con datos externos.

Links

Repositorio GitHub: <https://github.com/MartinSterli/TPI-Programacion>

Video demostrativo: <https://youtu.be/rkbbkRW2rQs>

Bibliografia / Webgrafia

Python Software Foundation. (2024). Python 3 Documentation. Recuperado de <https://docs.python.org/3/>

Python Software Foundation. (2024). csv - CSV File Reading and Writing. Recuperado de <https://docs.python.org/3/library/csv.html>

Python Software Foundation. (2024). Built-in Functions - sorted(). Recuperado de <https://docs.python.org/3/library/functions.html#sorted>

Real Python. (2024). Python Dictionaries. Recuperado de <https://realpython.com/python-dicts/>

Real Python. (2024). Reading and Writing CSV Files in Python. Recuperado de <https://realpython.com/python-csv/>

